

Main.java

+

Console

I/O

AI Agent

860 ms ✓

```
1 class Animal {
2     String som() { return "..."; }
3 }
4 class Cachorro extends Animal {
5     String som() { return "au"; }
6 }
7 public class Main {
8     public static void main(String[] args) {
9         Animal x = new Cachorro();
10        System.out.println(x.som());
11    }
12 }
13
14 // (a) Saída: au
15 // (b) Execução: método de instância sobrescrito; a JVM escolhe pelo
16 //     tipo real do objeto (Cachorro), não pelo declarado (Animal).
```

au

Main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  struct A {
5      void f() { cout << "A"; }
6  };
7  struct B : A {
8      void f() { cout << "B"; }
9  };
10
11 int main() {
12     B b; A *p = &b;
13     p->f();
14     return 0;
15 }
16
17 // (a) Saída: A
18 // (b) Compilação: sem 'virtual', a chamada é resolvida pelo tipo do
19 //     ponteiro (A*). Com 'virtual void f()' em A, a saída seria B e a
20 //     decisão passaria para a execução (vtable).
```

Console

I/O

AI Agent

425 ms

A

Main.java

```

1  class A { String nome = "A";
2      String getNome() { return nome; } }
3  class B extends A { String nome = "B";
4      String getNome() { return nome; } }
5
6  public class Main {
7      public static void main(String[] args) {
8          A x = new B();
9          System.out.println(x.nome + " " + x.getNome());
10     }
11 }
12
13 // (a) Saída: A B
14 // (b) x.nome -> compilação: atributos não são polimórficos (hiding),
15 //     vale o tipo declarado A.
16 //     x.getNome() -> execução: método sobrescrito, roda a versão de B.
    
```

Console

I/O

AI Agent

923 ms

A B

main.py × +

Console

I/O

AI Agent

128 ms ✓

```
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador(); b = Contador()
8 print(a.id, b.id, a.total)
9
10 # (a) Saída: 1 2 2
11 # (b) Execução: 'total' é atributo de classe, compartilhado pelas
12 #     instâncias. a.total não existe na instância, então a busca sobe
13 #     para a classe em tempo de execução.
```

1 2 2

Mainjava

Console

I/O

AI Agent

925 ms

```
1 class A {
2     static String quem() { return "A"; }
3 }
4 class B extends A {
5     static String quem() { return "B"; }
6 }
7 public class Main {
8     public static void main(String[] args) {
9         A x = new B();
10        System.out.println(x.quem());
11    }
12 }
13
14 // (a) Saída: A
15 // (b) Compilação: métodos static não são sobrescritos (hiding);
16 //     resolve pelo tipo declarado A, o objeto nem é consultado.
```

A

main.go

```
1 package main
2
3 import "fmt"
4
5 type Animal struct{}
6
7 func (Animal) Som() string { return "..."}
8 func (a Animal) Falar() string {
9     return "faz " + a.Som()
10 }
11
12 type Cao struct{ Animal }
13
14 func (Cao) Som() string { return "au"}
15
16 func main() {
17     fmt.Println(Cao{}.Falar(), Cao{}.Som())
18 }
19
20 // (a) Saída: faz ... au
21 // (b) Compilação: Go não tem herança, só embutimento. Falar() é
22 //     promovido de Animal e, lá dentro, a.Som() chama Animal.Som
23 //     (sem despacho virtual). Polimorfismo em Go só via interfaces.
```

Console

I/O

AI Agent

320 ms

faz ... au